

Implementación de servicios RESTful (II)

El objetivo de esta práctica es aprender a conectar nuestro servicio RESTful a una base de datos y aprender a gestionar los distintos tipos de excepciones que podemos encontrarnos. Para ello, partiremos del siguiente proyecto plantilla: <https://github.eii.us.es/aiss-us/lab7-implementing-restapi2-template>. Comprobaremos el correcto funcionamiento de la API utilizando Postman.

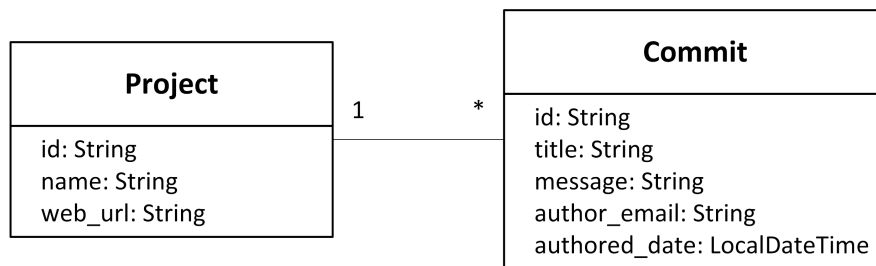


Ilustración 1: Modelo de datos

La API a desarrollar en esta práctica es una extensión de la creada en la práctica anterior (es recomendable partir del mismo proyecto y modificarlo). De nuevo, contamos con dos recursos: Project y Commit. Las operaciones a implementar en esta práctica son las mostradas en las tablas inferiores.

Las operaciones serán extendidas añadiendo nuevos códigos de respuesta relativos a errores de cliente (4XX). Asimismo, las operaciones sobre el recurso Commit han sido modificadas para que los commits formen siempre parte de un proyecto.

Recurso Project

HTTP	URI	Descripción
GET	/api/projects	Lista todos los proyectos de la aplicación.
GET	/api/projects/{projectId}	Devuelve el proyecto con id=projectId. Si el proyecto no existe, la API devuelve un error "404 not Found"
POST	/api/projects	Añade un nuevo proyecto cuyos datos se pasan en el cuerpo de la petición en formato JSON (no se debe pasar el id, se genera automáticamente). Si el proyecto se añade satisfactoriamente, devuelve "201 Created" y el contenido del proyecto, que por defecto contiene cero commits. Si alguno de los datos pasados por parámetro es inválido, la API

		devuelve un error “400 Bad Request”.
PUT	/api/projects/{projectId}	Actualiza el proyecto con id=projectId con los datos que se pasan en el cuerpo de la petición en formato JSON. Si el proyecto se actualiza correctamente, la API devuelve “204 No Content”. Si el proyecto no existe, la API devuelve un error “404 not Found”. Si alguno de los datos pasados por parámetro es inválido, la API devuelve un error “400 Bad Request”.
DELETE	/api/projects/{projectId}	Elimina el proyecto con id=projectId. Si el proyecto se elimina correctamente, la API devuelve “204 No Content”.

Cada proyecto tiene un *identificador*, *nombre*, *url* y una *lista de commits*. La representación JSON del recurso es:

```
{
  "id": "p1",
  "name": "Project name",
  "web_url": "www.project.com",
  "commits": [
    {
      "id": "c1",
      "title": "Commit title 1",
      "message": "Commit message 1",
      "author_email": "author1@gmail.com",
      "authored_date": "2022-12-19T10:10:00"
    },
    {
      "id": "c2",
      "title": "Commit title 2",
      "message": "Commit message 2",
      "author_email": "author2@gmail.com",
      "authored_date": "2021-04-06T12:30:00"
    }
  ]
}
```

Recurso Commit

HTTP	URI	Descripción
<u>GET</u>	/api/projects/{projectId} /commits	Lista todos los commits del proyecto con id=projectId. Si el proyecto no existe, la API devuelve un error “404 not Found”.
GET	/api/commits /{commitId}	Devuelve el commit con id=commitId. Si el commit no existe, la API devuelve un error “404 not Found”.

<u>POST</u>	/api/projects/{projectId} /commits	Añade un nuevo commit al proyecto con id=projectId. Los datos del nuevo commit se pasan en el cuerpo en formato JSON. Si el commit se añade satisfactoriamente, devuelve "201 Created" y el contenido del commit. Si el proyecto no existe, la API devuelve un error "404 not Found". Si alguno de los datos pasados por parámetro es inválido, la API devuelve un error "400 Bad Request".
PUT	/api/commits/ {commitId}	Actualiza el commit con id=commitId con los datos que se pasan en el cuerpo de la petición en formato JSON. Si el commit se actualiza correctamente, la API devuelve "204 No Content". Si alguno de los datos pasados por parámetro es inválido, la API devuelve un error "400 Bad Request".
DELETE	/api/commits/ {commitId}	Elimina el commit con id=commitId. Si el commit se elimina correctamente, la API devuelve "204 No Content".

Cada **commit** tiene un *identificador*, *título*, *mensaje*, *email del autor* y *fecha*. La representación JSON del recurso es:

```
{
  "id": "c1",
  "title": "Commit title",
  "message": "Commit message",
  "author_email": "author@gmail.com",
  "authored_date": "2022-12-19T10:10:00"
}
```

Ejercicios

Ejercicio 1. Conexión con base de datos

- a) Modifique el archivo `src/main/resources/application.properties` añadiendo la información necesaria para configurar la base de datos.
- b) Modifique los modelos de la API para indicar que deben ser considerados entidades de la base de datos. No olvide cambiar el datatype de los ids a long y eliminarlos del constructor.
- c) Reemplace los repositorios creados en la práctica anterior por interfaces que extiendan a `JpaRepository`.
- d) Actualice los controladores para que utilicen los métodos de los nuevos repositorios. No olvide modificar la funcionalidad de las operaciones sobre el recurso `Commit`, como se describe en las tablas superiores.

Ejercicio 2. Gestión de excepciones: Validación de inputs

- a) Modifique los modelos de datos para que se cumplan las siguientes restricciones:
 - El nombre de un proyecto no puede ser un string vacío.
 - El título de un commit no puede ser un string vacío.
 - El mensaje de un commit no puede ser nulo.
 - El email del autor de un commit debe ser un email válido.
 - La fecha de un commit debe estar en pasado.
- b) Cree un paquete `exception` y añada una clase `GlobalExceptionHandler` que contenga un método que gestione las excepciones de tipo `MethodArgumentNotValidException`. Este método permite devolver al usuario una lista de los errores de validación.
- c) Compruebe que la validación funciona correctamente utilizando Postman. Para ello, cree tests automáticos en aquellas requests que pueden devolver errores de validación. Dichos tests deben comprobar lo siguiente:
 - El código de respuesta es 400.
 - La respuesta devuelta por la API contiene los mensajes de validación que hemos definido.

Ejercicio 3. Gestión de excepciones: Recurso no encontrado

- a) Defina dos nuevas excepciones dentro del paquete `exception`: `ProjectNotFoundException` y `CommitNotFoundException`. Ambas deben devolver un código 404 y un mensaje que especifique que no se ha podido encontrar el recurso.
- b) Añada las excepciones a las operaciones definidas en los controladores según corresponda. Las tablas que describen la funcionalidad de la API especifican cuáles son las operaciones que pueden devolver un código 404.
- c) Compruebe que dichas operaciones devuelven un código 404 utilizando Postman. Para ello, cree tests automáticos que comprueben que las operaciones devuelven un código 404 al introducir un id que no existe.